

Aula React Native + Expo Notifications

Esta aula explica como utilizar notificações locais no React Native com Expo. O projeto também demonstra: Deep Linking Expo Router Listeners de notificação Permissões do sistema Navegação ao tocar na notificação

1. Importações

```
import * as Linking from 'expo-linking';
import * as Notifications from 'expo-notifications';
import { router } from 'expo-router';
```

Bibliotecas utilizadas: **expo-notifications**: gerenciamento de notificações. **expo-linking**: abertura de URLs. **expo-router**: navegação entre telas.

2. Configuração da notificação

```
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: true,
    shouldSetBadge: false,
    shouldShowBanner: true,
    shouldShowList: true,
  }),
});
```

Configuração	Função
shouldShowAlert	Mostra alerta
shouldPlaySound	Toca som
shouldSetBadge	Atualiza badge
shouldShowBanner	Mostra banner
shouldShowList	Mostra central de notificações

3. Deep Linking

```
const handleDeepLink = (url: string) => {
  if (url.startsWith('notify://')) {
    const path = url.replace('notify://', '/');
    router.push(path as any);
  } else {
    Linking.openURL(url);
  }
};
```

Fluxo: Verifica se é deep link interno. Converte notify:// em rota. Abre tela interna com router.push(). Se for URL externa, abre navegador.

4. useEffect

```
useEffect(() => {
}, []);
```

Executa apenas uma vez quando o app inicia. Muito utilizado para: Permissões Listeners Inicializações

5. Solicitação de permissão

```
const { status } = await Notifications.requestPermissionsAsync();
```

Abre o popup do sistema solicitando permissão para enviar notificações.

```
if (status !== 'granted') {
```

```
    alert('Você precisa liberar as notificações!');
  }
```

6. Listener da notificação

```
const subscription =
  Notifications.addNotificationResponseReceivedListener(response => {
    const data = response.notification.request.content.data;

    if (data.url && typeof data.url === 'string') {
      handleDeepLink(data.url);
    }
  });
```

Esse listener escuta quando o usuário toca na notificação.

7. Cleanup do listener

```
return () => subscription.remove();
```

Remove o listener quando o componente desmonta. Evita memory leaks e múltiplos listeners.

8. Agendamento da notificação

```
await Notifications.scheduleNotificationAsync({
  content: {
    title: "Olá! 🍌",
    body: 'Esta é uma notificação local do Expo',
    data: { url: 'https://www.google.com' },
  },
  trigger: {
    type: Notifications.SchedulableTriggerInputTypes.TIME_INTERVAL,
    seconds: 2
  },
});
```

Essa função cria uma notificação local.

Partes importantes: **title**: título da notificação. **body**: mensagem. **data**: dados extras escondidos. **trigger**: quando a notificação será enviada.

9. Trigger TIME_INTERVAL

```
trigger: {
  type: Notifications.SchedulableTriggerInputTypes.TIME_INTERVAL,
  seconds: 2
}
```

A notificação será disparada após 2 segundos.

10. Interface React Native

```
<Button
  title="Enviar Notificação em 2s"
  onPress={scheduleNotification}
/>
```

Ao clicar no botão: Uma notificação é agendada. Após 2 segundos ela aparece. Ao tocar nela, o app executa o deep link.

Resumo Final

expo-notifications gerencia notificações. **requestPermissionsAsync()** pede permissão. **scheduleNotificationAsync()** agenda notificações. **addNotificationResponseReceivedListener()** escuta cliques. **Linking.openURL()** abre URLs externas. **router.push()** navega internamente. **Deep Linking** conecta

notificações e navegação.